FOOD CHAIN DATA ENGINEERING PROJECT

Databricks Lakehouse • Medallion Architecture • CI/CD • Incremental Automation

Prepared by Mohit Taneja

1. Project Overview

The Food Chain Project is an end-to-end data engineering pipeline built on Databricks Lakehouse, designed to process raw operational data from a food-supply/retail business (customers, products, pricing, and orders) into clean, analytics-ready tables. The project follows the Medallion Architecture (Bronze → Silver → Gold), is version-controlled on GitHub, deployed through a Dev → Prod CI/CD process using Databricks Asset Bundles (DABs), and runs as a fully automated, incrementally-loading job that can be scheduled daily or weekly.

The pipeline currently orchestrates four dependent tasks — customer, product, and price dimension processing, followed by an incremental fact table load for orders — as shown in the Databricks Job Run graph below.

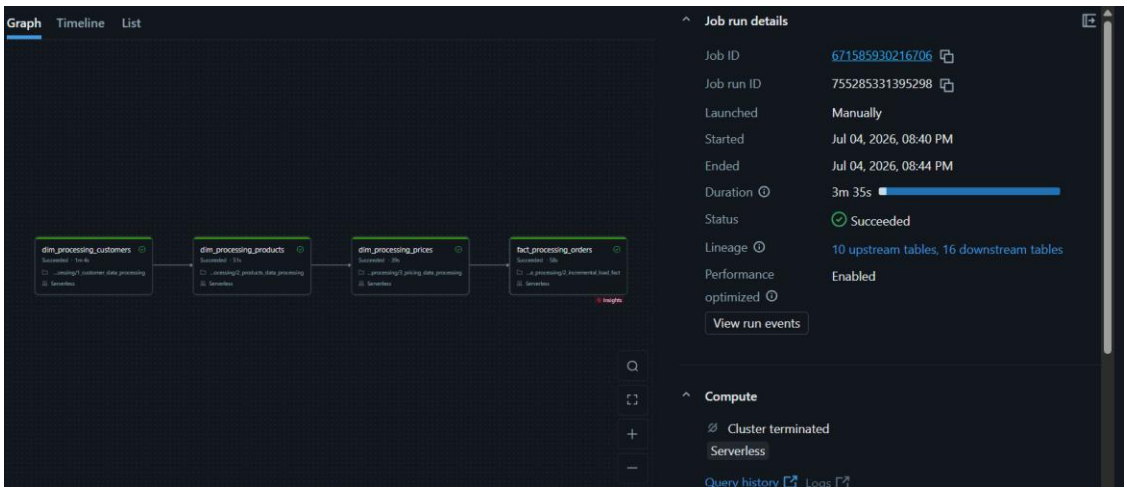

Figure 1: goods_incremental_pipeline_project — Job Run Graph (Succeeded, Serverless compute, 3m 35s)

2. Medallion Architecture

Data moves through three progressively refined layers inside Unity Catalog:

- **Bronze (Raw):** Raw ingested files/tables for customers, products, pricing, and orders, landed as-is from source systems with minimal transformation, preserving history for traceability.
- **Silver (Dimension/Cleansed):** Cleaned, deduplicated, and conformed dimension tables — dim_processing_customers, dim_processing_products, dim_processing_prices — with data-quality rules, type casting, SCD handling, and surrogate keys applied.
- **Gold (Fact/Business):** Curated, business-ready fact_processing_orders table, joined against the dimension layer, incrementally merged, and optimized for BI consumption (Power BI / Tableau / SQL analytics).

This layered design isolates raw ingestion issues from business logic, allows independent reprocessing of any layer, and gives lineage that Databricks' Job Run panel already surfaces (10 upstream tables, 16 downstream tables in Figure 1).

3. Pipeline Logic — Task-by-Task

The job goods_incremental_pipeline_project runs four notebook tasks in strict dependency order, each parameterised by catalog and data_source so the same notebook logic can be reused across environments and datasets:

Order	Task Key	Purpose	Depends On
1	dim_processing_customers	Cleans & loads the Customers dimension (dedupe, standardize address/contact fields, generate surrogate keys).	—
2	dim_processing_products	Cleans & loads the Products dimension (categorization, SKU standardization).	dim_processing_customers
3	dim_processing_prices	Processes gross pricing data, applies pricing history/versioning logic.	dim_processing_products
4	fact_processing_orders	Incrementally merges new/changed Orders into the Gold fact table using the cleaned dimensions for key lookups.	dim_processing_prices

Running the dimensions before the fact table guarantees that every foreign key referenced by an order (customer, product, price) already exists in Silver before the Gold merge executes — preventing orphaned records and join mismatches.

4. CI/CD Pipeline — Dev → Prod Promotion

Deployment is managed through Databricks Asset Bundles (DABs), using a declarative Python job definition (Job.from_dict) that is checked into Git and deployed through separate Dev and Prod targets — the same code, different workspace/catalog bindings, so nothing is ever hand-edited directly in Prod.

4.1 Environments

Environment	Workspace Folder	Purpose
Dev	/…/folders/3189776823685914	Feature development, unit testing, dry-run job execution, data-quality validation on sample/dev catalog (goods_ct dev schema).
Prod	/…/folders/3189776823685916	Production job execution against the live catalog, on a schedule, with email alerting.

4.2 Recommended Pipeline Flow

1. Developer pushes notebook/bundle changes to a feature branch and opens a Pull Request against main on GitHub.
2. CI stage (GitHub Actions): lint notebooks, run 'databricks bundle validate', and execute unit/data-quality tests against the Dev catalog.

3. On merge to main, GitHub Actions runs 'databricks bundle deploy --target dev' and 'databricks bundle run goods_incremental_pipeline_project --target dev'.
4. Automated checks confirm the Dev run status is Succeeded, row counts/DQ checks pass, and no schema drift is detected.
5. A manual approval gate (GitHub Environments 'production' protection rule) requires sign-off before promotion.
6. On approval, the same bundle is deployed to Prod: 'databricks bundle deploy --target prod', pointing tasks at the Prod workspace folder and the production catalog/schema.
7. Post-deploy smoke test triggers one Prod run manually; only after it succeeds is the job's schedule/trigger enabled for unattended runs.
8. Any failure at Dev or Prod halts promotion automatically and notifies the team by email (on_failure notification already configured in the job).

4.3 Example databricks.yml (Bundle) Targets

```
targets:
  dev:
    workspace:
      host: https://dbc-a622cbe7-e6bb.cloud.databricks.com
      root_path: /Workspace/Users/mohittaneja222@gmail.com/1_dev_food_chain_project
    variables:
      catalog: goods_ct_dev
  prod:
    mode: production
    workspace:
      host: https://dbc-a622cbe7-e6bb.cloud.databricks.com
      root_path: /Workspace/Users/mohittaneja222@gmail.com/2_prod_food_chain_project
    variables:
      catalog: goods_ct
```

The job definition itself (goods_incremental_pipeline_project.py, shown below) stays identical across targets — only the 'catalog' base_parameter and notebook root path change between Dev and Prod, which is exactly what DAB target variables are for.

4.4 Job Definition (Reference)

```
from databricks.bundles.jobs import Job
goods_incremental_pipeline_project = Job.from_dict({
  "name": "goods_incremental_pipeline_project",
  "email_notifications": {
    "on_start": [...], "on_success": [...], "on_failure": [...]
  },
  "tasks": [
    { "task_key": "dim_processing_customers", "notebook_task": {...} },
    { "task_key": "dim_processing_products", "depends_on": [...] },
    { "task_key": "dim_processing_prices",  "depends_on": [...] },
    { "task_key": "fact_processing_orders", "depends_on": [...] }
  ],
  "queue": { "enabled": True },
```

```
  "performance_target": "PERFORMANCE_OPTIMIZED"
})
```

5. Incremental Load Automation

The fact_processing_orders task is built as an incremental (not full-refresh) load, so every run only processes new or changed order records rather than reprocessing the entire history:

- A watermark column (e.g., last_updated_ts or order_id high-water-mark) is tracked in a control/checkpoint table.
- Each run reads only Bronze records newer than the last successful watermark.
- A Delta Lake MERGE INTO statement upserts matched records and inserts new ones into the Gold fact table, keyed on the order's business key.
- Delta Lake's ACID transactions and schema enforcement guarantee the merge is safe to re-run (idempotent) if a job is retried after a transient failure.
- The watermark/checkpoint only advances after a successful commit, so a failed run never skips or double-counts data.

Because dimension tasks run first in the same job, the incremental fact merge always resolves against the freshest customer, product, and price keys — keeping Gold consistent without needing a full dimensional reload.

6. Scheduling — Daily / Weekly Automation

Once validated in Prod, the same job is attached to a Databricks Trigger so it runs unattended on a fixed cadence instead of manually:

```
"trigger": {
  "pause_status": "UNPAUSED",
  "periodic": { "interval": 1, "unit": "DAYS" }   // daily
}
```

For a weekly cadence, the same block can instead use a cron-based schedule, for example:

```
"schedule": {
  "quartz_cron_expression": "0 0 3 ? * MON",      // every Monday 03:00
  "timezone_id": "Asia/Kolkata",
  "pause_status": "UNPAUSED"
}
```

Both settings live in the same bundle YAML/Python job definition, so switching cadence is a one-line config change deployed through the same Dev → Prod pipeline described above — never a manual click in the UI. The existing on_start, on_success, and on_failure email notifications keep the team informed of every scheduled run without any extra setup.

7. Monitoring & Observability

- Job Run graph (Figure 1) gives a visual DAG of task status, duration, and dependency order for every run.
- Lineage panel (10 upstream tables, 16 downstream tables) confirms exactly which tables feed and consume this pipeline — useful for impact analysis before schema changes.
- Serverless compute means no cluster management — Databricks provisions and tears down compute automatically per run (visible as 'Cluster terminated' after completion).
- Performance Optimized target plus job queueing keeps runs fast and prevents overlapping concurrent runs of the same job.

8. Environment & Repository Links

Production Workspace:

Development Workspace:

GitHub Repository: https://github.com/mohittaneja786/food_chain_project

GitHub README: https://github.com/mohittaneja786/food_chain_project/blob/main/README.md

9. Summary

In short: raw data lands in Bronze, is cleansed into Silver dimension tables (customers → products → prices, in that dependency order), and is incrementally merged into a Gold fact table for orders. The entire job is defined once as code, deployed identically to Dev and Prod through Databricks Asset Bundles, gated by validation and manual approval before production promotion, and finally scheduled to run unattended on a daily or weekly cadence — with email alerts covering start, success, and failure at every run.